

The Plethora of Problems With Profiles

Document #: P3586R0 [[Latest](#)] [[Status](#)]
Date: 2025-01-13
Project: Programming Language C++
Audience: Evolution Working Group
Reply-to: Corentin Jabot
<corentin.jabot@gmail.com>

§ 1 Introduction

Profiles ([\[P3081R1\]](#)) aim to be a one-size-fits-all solution to fix three different classes of problems.

- Some undefined behavior could be checked at runtime with (arguably) some performance cost, and we would like tools to inject such checks.
- Some constructs are easy to misuse and could lead to unsafeties, and we would like to detect these constructs and error on them (for example, narrowing casts).
- Some memory unsafeties could be semi-reliably detected through compile time control flow analysis.

However, these classes of problems should be considered independently, as they are subject to widely different constraints and design considerations.

§ 2 Runtime checks

As explained in [\[P3543R0\]](#), runtime checks, contracts, and, to some extent, erroneous behavior all try to solve the same problem, and a solution for runtime checks should, therefore, piggyback on contracts, regardless of any perceived time pressure or deadline.

At the same time, [\[P3081R1\]](#) does not explore the proposal's impact on existing widely deployed solutions such as sanitizers.

It would make sense to come to a common understanding and a cohesive set of features. The contract study group should be consulted on the path forward and has been designing towards this end goal for quite a while.

Therefore, the rest of the paper will focus on other aspects of the profiles proposal, mostly the general design and how it deal with dangerous syntactic constructs.

The lifetime analysis, as well as the recently proposed union handling are not considered in this paper, as neither of these ideas (which would benefit from deployment experience) are mature enough to be fairly considered. However, Sean Baxter wrote an excellent, well researched piece on the [short comings of the lifetime profile](#).

§ 3 Ruminations on profile opt-in mechanisms

After many iterations, [\[P3081R1\]](#) is proposing that a profile can be opt-in on a per-TU basis through an attribute (`[[profiles::enforce]]`).

The attribute syntax is problematic as it is currently accepted by implementations, [which will gladly ignore it](#). Note that this is not a philosophical question about the ignorability of attributes. The fact of the matter is that older toolchains will ignore the annotation and can't be changed. Safety features should not be ignorable; allowing them to be will lead to vulnerabilities.

That mechanism interacts poorly with existing headers, which must be assumed incompatible with any profiles. [\[P3081R1\]](#) recognizes that and suggests - That standard library headers are exempt from profile checking. - That other headers may be exempt from profile checking in an implementation-defined manner.

It is not clarified whether such exemptions apply to template instantiations involving user-defined types and how these exemptions would not promote vulnerabilities.

The standard library carve-out is yet another demonstration of how it is problematic to try to apply the same tool to lexical constructs and runtime behavior alike. Indeed, it might be reasonable not to error on the presence of `reinterpret_cast` within the STL; however, would we really want to disable preconditions, even when the preconditions would check user-provided values? This would do nothing to improve safety.

At the time of writing, module support is also not mentioned. Does the `[[profiles::enforce]]` attribute apply to the GMF? Private partitions? How would profiles impact ODR rules and importable headers?

And then, there is `[[profiles::apply]]` - which produces warnings instead of errors. How does that work in any non-toy software?

Surely, the control of warnings is the responsibility of the final application and not its component libraries. Depending of one's warning and error policy, is the intent to edit a bunch of source files to turn warnings into errors? Or use macros?

Again, that kind of behavior is best left to implementations. The committee's time is better spent identifying dangerous constructs and offering replacements when possible rather than deciding if something should be a warning, a fix-it, or how it interacts with `-Werror` and the many diagnostic-controlling flags and features offered by implementations.

§ 4 Ruminations on profile opt-out mechanisms

`[[profiles::suppress]]` fails the [ignorability of attributes](#) test. This is less problematic than for `enforce` but it has not been discussed.

It's also very verbose and unergonomic for something that will have to be used relatively frequently. (compared to Rust's `unsafe{}` blocks, for example). It is very hard to qualify the usability of this mechanism without usage experience.

It is also unclear that all rules should be suppress-able, when a better alternative exists (for example, `narrow_cast`).

§ 5 Profiles don't have a good backward compatibility story.

[\[P3081R1\]](#) proposes categorizing profiles by kind of errors (type, numeric, memory, etc.). Generally, users would expect cheap static checks, expansive static analysis, and runtime checks to be controlled by separate flags. There is a reasonable expectation that the runtime performance and the compile time cost of a feature remains reasonably constant over time.

As such that categorization (that mixes in a single profile different kinds of checks), along with the fact that there is no description of how profiles evolve and inter-operate in the long run, means that profiles will not have a great backward-compatibility story.

§ 6 Preventing dangerous constructs.

[\[P3081R1\]](#) identifies some dangerous constructs and proposes to make them conditionally ill-formed (or to force an implementation to warn on them, more on that later).

- `reinterpret_cast`
- `const_cast`
- static downcasts
- some C style casts
- Pointer arithmetic
- narrowing casts
- `delete`
- `va_arg`
- Array decay

— ...

Not all of these constructs are safety issues for the same reasons.

For example, static down casts and narrowing casts are dangerous mostly because they are not clearly visible in the source. P3081R1 proposes a `narrow_cast` replacement function (this is great). However, no replacement is proposed for static downcasts. Yet, static downcasts are necessary to implement CRTP and “homegrown” dynamic casting (LLVM’s `dyn_cast`, Qt’s `qobject_cast`). So, static downcast probably needs an explicit replacement - maybe one with preconditions.

`reinterpret_cast` is an obvious foot gun. However, its spelling makes that clear. Do we want to encourage all usages of `reinterpret_cast` to be replaced by `[[profiles::suppress(type_safety)]] reinterpret_cast`? What do we gain besides making users less attentive to the code they write?

`const_cast<const T>` is always acceptable, and whether it should be banned for safety reasons is unclear. Should stripping away `const` be banned or should we research ways to detect mutation of actually `const`s objects/subobjects? P3081R1 fails to be clear about the impact of this proposal. `const_cast` is often in code bases that do not have `const`-correct-interfaces for legacy reasons. Here, too, we should avoid forcing users to write `[[profiles::suppress(type_safety)]] const_cast<>` everywhere. The safety implications of an invalid `const_cast` have also not been explored. Is that UB exploitable in such a way that it would be a safety concern?

Array decay is, unlike other features in that list, not a lexical construct but a behavior of the language. Whether that behavior can be changed needs much more work to determine impact and viability.

§ 6.1 Should `delete` be ill-formed?

In its latest iteration [P3081R1] proposes to make `delete` and `free` rejected by the lifetime profile. Presumably (the paper offers no motivation), the intent is to discourage manual memory management. However, allowing `new` and not `delete` could encourage bad practices and resources leak (which, while not UB could be a vulnerability) Rust considers allocations an unsafe operation, which avoids that issue.

At the same time, rust also consider deferencing a raw pointer unsafe, whereas just considering `delete` unsafe would not materially help with use-after-free. The problem then moves to `unique_ptr<T>::get()`.

§ 6.2 What about constant evaluation?

[P3081R1] whether dangerous constructs that are not potentially used, such as when used only in constant or unevaluated context should be ill-formed (despite not causing safety issues).

§ 7 P3081R1 is detrimental to the quality of implementations

[P3081R1] tries to offer novel recommendations for implementations

- to offer specific fix-its (replacements)
- to offer specific warnings
- to behave in a specific way in the presence of `-Werror` or equivalent.

Ultimately, this is not very useful to implementers or end-users and is not a great use of committee time. Implementations have decades of experience with what constitutes a suitable, useful, actionable warning. At the same time, users have particular expectations regarding the behavior of things like `-Werror` (as well as diagnostic pragmas), etc.

Attempts to recommend different warning behaviors that would be counterintuitive to user expectations or that would not otherwise improve safety would just be ignored by implementations. `-Werror` will continue to behave as `-Werror` does.

Fix-it suggestions which are not universally applicable are also unlikely to be implemented (for example, replacing `static_cast` with `dynamic_cast` could be, depending on the specifics of the project, not viable or a terrible idea).

§ 8 P3081R1 mixes safety and stylistic concerns

[P3081R1] proposes, for example

- To mandate fix-it diagnostics for superfluous `dynamic_cast`
- To mandate fix-it replacement for `static_cast<T>(T);`
- To mandate fix-it replacement for functional casts

While these might be useful QoI behavior (often [implemented in static analyzers](#)), they do not expose security concerns and are harmless or opinionated stylistic choices that are best left to users implementers.

It is critical that any safety diagnostics only flags actually dangerous constructs to avoid users silencing the warnings. Mixing concerns would reduce any trust users may place in these diagnostics.

§ 9 Bound checking in the language is harmful and prevents adoptability.

One of the checks proposed by P3081R1 is to insert bound checks on any `operator[]` of an arbitrary type that looks like a sequence container.

As explained in [P3543R0], many vector-looking containers do not satisfy the semantic requirement of a vector-like container such that their `operator[]`, or `size()` member functions behave in interesting ways and trying to infer semantics from lexical properties is a surefire way to break a lot of existing applications.

Moreover, all standard implementations, Qt, folly, LLVM, abseil, and others widely deploy frameworks all have checked preconditions on their containers' `operator[]`, which would lead to duplicate checks - and shows that this is mostly a solved problem.

In fact, while writing this paper, `libstdc++` enabled bound checkings and other preconditions by default.

Once [P2900R0] is adopted, these will all consolidate over time into precondition assertions, and the user story will only get better.

§ 10 A profile by any other name would still be a subsetting mechanism.

An enforced profile can make valid C++23 ill-formed. Therefore, profiles are a subsetting mechanism. A dialect, if you will. And that's perfectly fine. It is not particularly useful to pretend otherwise or to pretend that we will never want to break dangerous constructs over time.

In fact, in the context of safety, it will be desirable or necessary for dangerous constructs (implicit narrowing conversion, for example) to stop being well-formed at some point.

However, it is interesting that profiles create as many dialects as there are possible combinations of profiles (at least profiles that impact language constructs), and while there is a lot of experiences with C++ subsets and supersets (no exceptions/no RTTI/gnu extensions/etc.), it seems desirable to avoid a combinatorial explosion of restrictions and semantic changes as that would reduce the interoperability of libraries and the overall health of the C++ ecosystem.

At the same time, in their current forms, profiles do not appear to be a robust tool to subset the language. The interactions with modules, template instantiations, default arguments, ODR, etc, are simply not specified.

We clearly need a well-specified mechanism to offer a restricted set of features in some translation units.

§ 10.1 The Age of Epochs

Epochs ([P1881R1]) tried to define such a solution and the paper already proposed safety-related restrictions.

Similarly, the visionary [D0997R3] proposed to remove some constructs from modules.

Modules, which constitute complete translation units, offer a natural boundary for the subsetting of the language and resolve some of the ODR-related concerns that arise by allowing different subsets and language rules to exist at different points of the same translation units.

Note that, should the standard define epochs, implementations could offer flags to opt-in non-module TUs to a given epoch, which would help with adoptability.

During the initial presentation of Epochs, concerns were raised regarding the interaction with templates.

However, if we limit epochs to making dangerous semantics ill-formed, and we specify that the rules of an epoch equally apply to specializations of declarations attached to a given epoch and their default arguments/parameters, we can come up with a reasonable model. Note that these questions apply equally to profiles and need to be solved regardless.

Whether epochs should be allowed to have valid but different semantics from one another is unclear. However, how much can be done with epochs doesn't need to be answered initially as long as we can make them a reliable and predictable tool to make some constructs ill-formed, including many of the dangerous lexical constructs identified by the profile papers.

Compared to profiles, epochs would be better defined, non-ignorable, fully specified, and avoid the concerns of having multiple conflicting subsetting within the same TU.

At worst, C++ would have one epoch per cycle, which seems more manageable than a profile proliferation.

§ 11 Making progress on safety

The various profiles-related paper identify some sources of unsafeties and attempt to solve them all with a single tool. As profiles are an opt-in, ignorable feature, they do not meaningfully improve or make the language safer or more usable over time, nor do they solve the discoverability issues that current vendor tools might have. A lot of the profiles aim to replicate what is already done by implementers. Trying to standardize warnings or opined fix-its is unlikely to meaningfully improve the language in the long term. Profiles operate in an area that implementations and tools (both open-source and commercial) are actively exploring, researching, and innovating in.

That being said, it is evident that language safety should be an area of focus, and there would be a lot of value in finding ways to evolve the language by:

- Removing or deprecating dangerous constructs while providing more explicit replacements
- Consider an epoch-like mechanism if the removal of dangerous constructs proves hard to adopt
- Use contracts to detect some language UB at runtime as proposed in [\[P2900R0\]](#)
- Keep working with standard library implementations to enforce preconditions
- Keep working with implementations and tool providers on control flow analysis tools until these tools are sufficiently mature to decide whether any part should be standardized
- Explore long-term language solutions for memory safety.
- Explore whether the committee could help in the adoption of CHERI/MTE/pointer authentication.
- Explore ways to improve compatibility and interfaces with memory-safe languages, which are becoming the tools of choice in some industries.
- Not closing doors prematurely. Many ideas are worth considering, even if they are scary or ambitious.

C++’s safety story should involve an array of standard changes and vendor-provided solutions, applying the right tools to each problem

	Dangerous constructs	Language UB	Library UB	Memory/Lifetime/Thread Safeties
WG21	Depreciation Removal Replacement Epochs	Contracts Erroneous Behavior Safer alternatives	Contracts Unsafe functions coloring?	Research towards Safe C++? Non trivial relocation?
Vendors	Warnings Guidelines Static analysis	Sanitizers		Sanitizers Pointer Tagging Static analysis

§ 11.1 On trains and missing them

Safety is critical to the future of C++. However, we should not let a sense of urgency get the better of us. There are two scenarios to consider here:

- Either a given language-safety-related feature can be backported to older C++ versions, so it will (implementers care about safety, too!).
- It is C++26 or C++29-specific and, for some reason, can’t be backported as a conforming extension. In that case, given the low adoption rate of C++26 at this time, pushing it to early C++29 would have no impact.

It is also worth noting that nothing in profiles in their current form is normative enough that it couldn't be pursued as a standalone guideline outside of the standard and its release cycle. Concerns with language safety far predate our renewed interest, and regulatory pressure isn't a very good reason to try to solve a very complex and multi-faceted issue in a few weeks. There is going to be another train.

§ 12 Further reads

- [Story-time: C++, bounds checking, performance, and compilers](#)
- [Retrofitting spatial safety to hundreds of millions of lines of C++](#)
- [Eliminating Memory Safety Vulnerabilities at the Source](#)
- [Towards the next generation of XNU memory safety](#)
- [Compiler Options Hardening Guide for C and C++](#)
- [Getting the maximum of your C compiler, for security](#)
- [Clang-Tidy Checks](#)
- [Memory error checking in C and C++: Comparing Sanitizers and Valgrind](#)
- [Control Flow Integrity](#)
- [Safe Buffers](#)
- [Safe C++](#)
- [Clang IR](#)
- [Memory Tagging and how it improves C/C++ memory safety](#)
- [CHERI](#)
- [Pointer Authentication](#)
- [The Rustonomicon](#)
- [C++/Rust Interoperability Problem Statement](#)
- [Memory-Safety Challenge Considered Solved? An In-Depth Study with All Rust CVEs](#)
- [Ferocenne](#)
- [Stacked Borrows: An Aliasing Model for Rust](#)
- [Report to CISA - Technical Advisory Council, Memory Safety](#)
- [The Unexplored Terrain of Compiler Warnings](#)

§ 12.1 Videos

- [The existential threat against C++ and where to go from here](#)

- [Security in C++ - Hardening Techniques From the Trenches](#)
- [Embracing an Adversarial Mindset for C++ Security](#)
- [Safety and Security: The Future of C++](#)
- [Fixing C++ with Epochs](#)
- [Mitigating lifetime issues for C++20 coroutines](#)

§ 13 Acknowledgments

Thanks to David Ledger and Joshua Bern for reviewing drafts of this paper. Thanks to Erich Keane, Aaron Ballman, and Shafik Yagmour for insightful conversations and feedbacks.

§ 14 References

- [D0997R3] Draft 3 P0997 “Retire Pernicious Language Constructs in Module Contexts.”
<https://wg21.link/d0997r3>
- [P1881R1] Vittorio Romeo. 2020-01-12. Epochs: a backward-compatible language evolution mechanism.
<https://wg21.link/p1881r1>
- [P2900R0] Joshua Berne, Timur Doumler, Andrzej Krzemieński. 2023-09-27. Contracts for C++.
<https://wg21.link/p2900r0>
- [P3081R1] Herb Sutter. Core safety Profiles: Specification, adoptability, and impact.
wg21.link/P3081
- [P3543R0] Mungo Gill, Corentin Jabot, John Lakos, Joshua Berne, Timur Doumler. 2024-12-17. Response to Core Safety Profiles (P3081).
<https://wg21.link/p3543r0>